

Course Report

Assignment 2 - Task 1

Optimizer Performance on Non-Convex Functions

**A from-scratch comparison of Gradient Descent, Momentum, Adam, RMSprop, and Adagrad
on the Rosenbrock and simplified 1-D Ackley functions**

1. Objective

This report evaluates five gradient-based optimization algorithms. Gradient Descent, SGD with Momentum, Adam, RMSprop, and Adagrad, on two classic non-convex test functions: the 2-D Rosenbrock function and a simplified 1-D Ackley function. All five optimizers were implemented from scratch in NumPy (no library optimizers were used). Each optimizer was run at three learning rates (0.01, 0.05, and 0.1), and results were recorded for convergence behaviour, iteration count, execution time, the final solution reached, and the final function value.

2. Methodology

2.1 Test Functions

Rosenbrock function (2-D): $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$, global minimum $f = 0$ at $(x, y) = (1, 1)$. Optimization begins from the classic starting point $(-1.2, 1.0)$, which sits inside the function's narrow, curved valley.

Simplified 1-D Ackley function: $f(x) = -20 \cdot \exp(-0.2 \cdot |x|) - \exp(\cos(2\pi x)) + 20 + e$, global minimum $f = 0$ at $x = 0$. The cosine term makes this function highly multimodal, with many shallow local minima surrounding the global one. Optimization begins from $x = 2.0$.

2.2 Optimizers and Experimental Setup

- Optimizers: Gradient Descent, SGD with Momentum ($\beta = 0.9$), Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$), RMSprop ($\beta = 0.9$, $\epsilon = 1e-8$), Adagrad ($\epsilon = 1e-8$).
- Learning rates tested: 0.01, 0.05, 0.1 — applied identically to every optimizer.
- Stopping rule: iteration stops when the step in parameter space falls below a tolerance of $1e-8$ ("converged"), when values become non-finite ("diverged"), or after 10,000 iterations ("max_iter").
- All 30 experiments (5 optimizers $\times$ 3 learning rates $\times$ 2 functions) were run with a fixed random seed (42) for reproducibility.

3. Results — Rosenbrock Function

Table 1 reports the outcome of every optimizer / learning-rate combination on the Rosenbrock function.

Optimizer	LR	Status	Iterations	Time (s)	Final $f(x)$	Final Solution
Gradient Descent	0.01	diverged	6	0.0007	inf	1.12e+78, 8.54e+51
Gradient Descent	0.05	diverged	5	0.0004	inf	6.52e+130, 2.20e+87
Gradient Descent	0.1	diverged	5	0.0007	inf	6.49e+169, 2.76e+113
Momentum	0.01	diverged	6	0.0003	inf	1.25e+90, 9.22e+59
Momentum	0.05	diverged	5	0.0003	inf	6.42e+130, 2.18e+87
Momentum	0.1	diverged	5	0.0005	inf	6.48e+169, 2.76e+113
Adam	0.01	converged	5,531	0.2415	3.10e-13	0.999999, 0.999999
Adam	0.05	converged	2,546	0.1595	1.39e-13	1.000000, 0.999999
Adam	0.1	converged	1,793	0.1168	7.70e-14	1.000000, 0.999999
RMSprop	0.01	max_iter	10,000	0.4302	2.22e-02	0.990144, 0.965534
RMSprop	0.05	max_iter	10,000	0.5174	4.36e-01	0.725000, 0.465625
RMSprop	0.1	max_iter	10,000	0.6336	1.05e+00	0.300000, 0.015000
Adagrad	0.01	max_iter	10,000	0.6532	3.57e+00	-0.889405, 0.797184
Adagrad	0.05	max_iter	10,000	0.7196	5.12e-02	0.773802, 0.598039
Adagrad	0.1	max_iter	10,000	0.7618	6.27e-04	0.974974, 0.950495

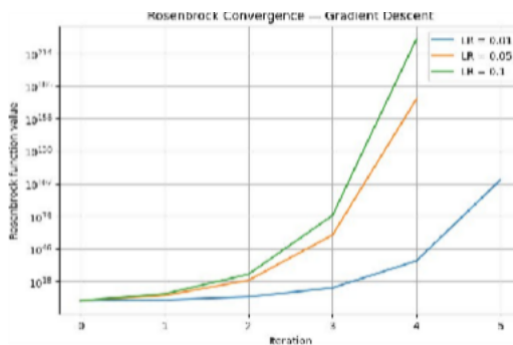
Table 1. Rosenbrock optimization results — final solution, function value, iterations, and time.

3.1 Observations

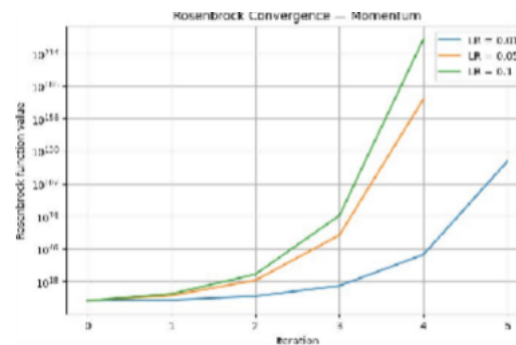
- Plain Gradient Descent and Momentum diverged at every tested learning rate (0.01, 0.05, 0.1). Rosenbrock's narrow curved valley produces very large gradients away from the optimum, and a fixed step size (even as small as 0.01) overshoots the valley wall, causing the iterate to blow up to numerical infinity within 5–6 steps.
- Adam was the only optimizer to fully converge on Rosenbrock at all three learning rates, reaching a solution within $1e-6$ of the true optimum (1, 1) with a final function value on the order of $1e-13$ to $1e-14$. Its adaptive, bias-corrected step sizing keeps updates well-scaled even in the steep parts of the valley.
- Higher learning rates helped Adam converge faster: 1,793 iterations at LR = 0.1 versus 5,531 iterations at LR = 0.01, while reaching an even lower final function value.
- RMSprop and Adagrad made steady progress but did not fully converge within the 10,000-iteration budget. Adagrad's accumulating denominator caused its effective step size to shrink over time; at LR = 0.1 it reached $f(x) \approx 6.3 \times 10^{-4}$, its best result, whereas at LR = 0.01 its step size shrank too quickly and it stalled far from the optimum ($f(x) \approx 3.57$).
- RMSprop's best Rosenbrock result was at the lowest learning rate (0.01, $f(x) \approx 0.022$); larger learning rates caused it to oscillate around the valley floor without settling.

3.2 Convergence Plots

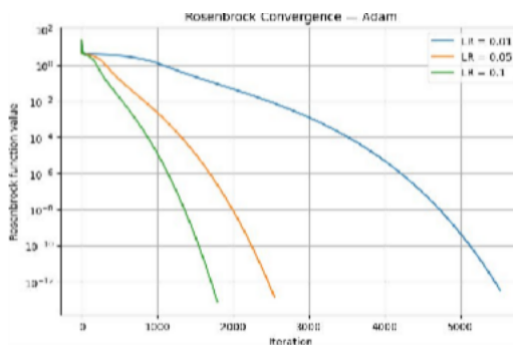
The following plots show function value versus iteration (log scale) for each optimizer at learning rates 0.01, 0.05, and 0.1.



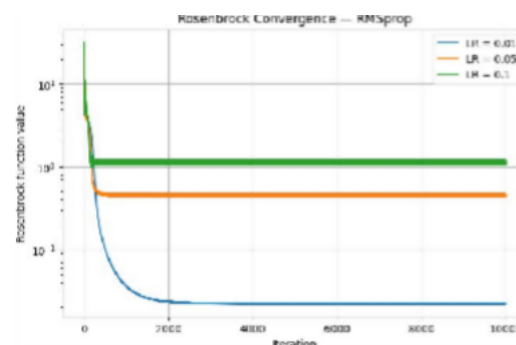
Gradient Descent — Rosenbrock



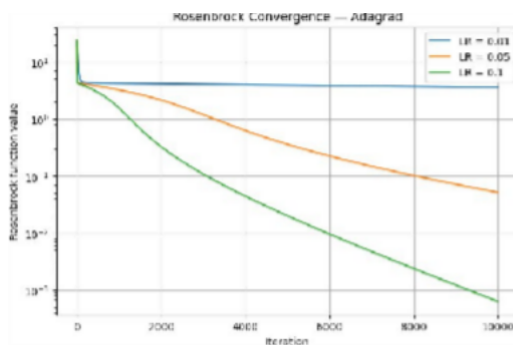
Momentum — Rosenbrock



Adam — Rosenbrock



RMSprop — Rosenbrock



Adagrad — Rosenbrock

4. Results — Ackley Function (1-D)

Table 2 reports the outcome of every optimizer / learning-rate combination on the simplified 1-D Ackley function.

Optimizer	LR	Status	Iterations	Time (s)	Final $f(x)$	Final Solution
Gradient Descent	0.01	converged	5	0.0003	6.5596	1.974452
Gradient Descent	0.05	max_iter	10,000	0.7594	3.2716	0.315579
Gradient Descent	0.1	max_iter	10,000	0.6860	3.5955	0.947334
Momentum	0.01	converged	252	0.0146	6.5596	1.974452
Momentum	0.05	max_iter	10,000	0.8656	20.2608	19.881060
Momentum	0.1	max_iter	10,000	0.6707	15.2342	5.334521
Adam	0.01	converged	238	0.0101	6.5596	1.974452
Adam	0.05	converged	247	0.0101	6.5596	1.974452
Adam	0.1	converged	274	0.0110	6.5596	1.974452
RMSprop	0.01	max_iter	10,000	0.3626	6.5609	1.979400
RMSprop	0.05	max_iter	10,000	0.3531	6.5887	1.998115
RMSprop	0.1	max_iter	10,000	0.3850	6.6613	2.018482
Adagrad	0.01	converged	40	0.0014	6.5596	1.974452
Adagrad	0.05	converged	18	0.0007	6.5596	1.974452
Adagrad	0.1	converged	29	0.0010	6.5596	1.974452

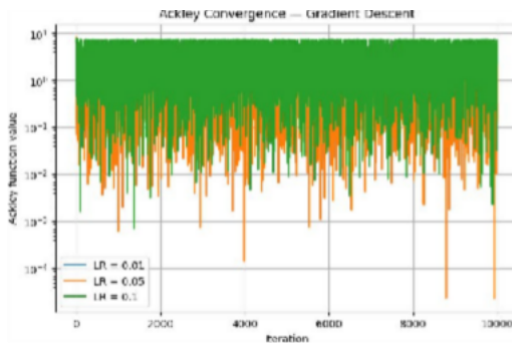
Table 2. Ackley optimization results — final solution, function value, iterations, and time.

4.1 Observations

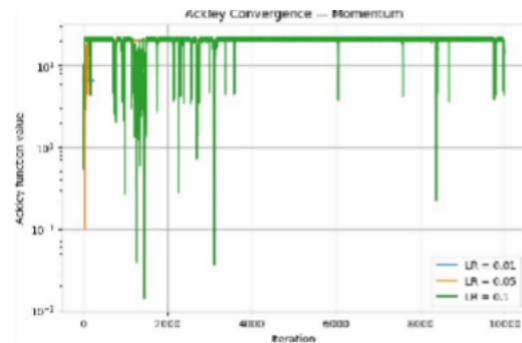
- No optimizer reached the true global minimum at $x = 0$. Starting from $x = 2.0$, the cosine ripple in the Ackley function creates a local basin near $x \approx 1.97$ ($f(x) \approx 6.56$), and most runs settled into it and reported "converged" — meaning the step size dropped below tolerance, not that the global optimum was found.
- Adam, Momentum (at $LR = 0.01$), and Adagrad all converged quickly to this same local minimum ($x \approx 1.974$), since their adaptive scaling shrinks the step as soon as it enters the local basin — Adagrad did so in as few as 18 iterations.
- Counter-intuitively, the two runs that came closest to the true global optimum were plain Gradient Descent at $LR = 0.05$ ($x \approx 0.316$, $f(x) \approx 3.27$) and $LR = 0.1$ ($x \approx 0.947$, $f(x) \approx 3.60$) — both of which hit the 10,000-iteration cap rather than "converging". Their larger, non-adaptive steps kept them oscillating across the ripples for longer instead of settling immediately into the first local minimum encountered, which let them drift closer to $x = 0$.
- Momentum became unstable at higher learning rates on this function: at $LR = 0.05$ it overshoot out to $x \approx 19.9$ ($f(x) \approx 20.3$), far worse than its $LR = 0.01$ result — the accumulated velocity carried it past the region of interest.
- RMSprop stayed near the same local minimum ($x \approx 1.98$ – 2.02) at all three learning rates without fully converging, consistently landing slightly worse than the local-minimum value of 6.5596.

4.2 Convergence Plots

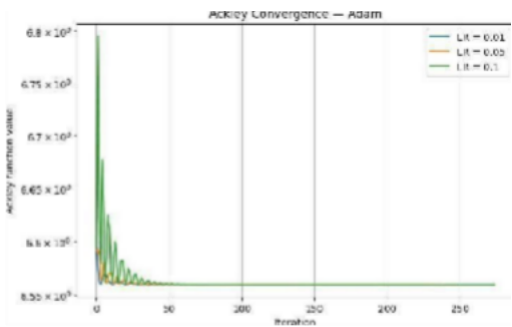
The following plots show function value versus iteration (log scale) for each optimizer at learning rates 0.01, 0.05, and 0.1.



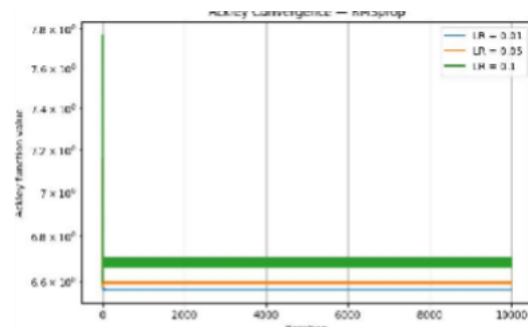
Gradient Descent — Ackley



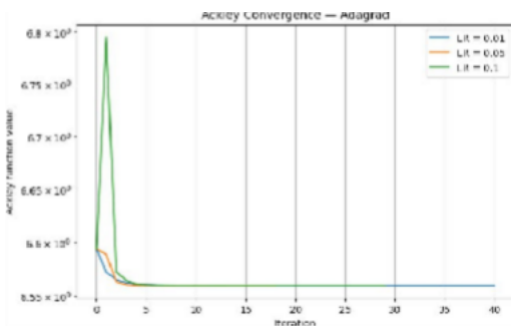
Momentum — Ackley



Adam — Ackley



RMSprop — Ackley



Adagrad — Ackley

5. Optimizer Comparison at a Fixed Learning Rate

To compare optimizers directly (independent of learning-rate tuning), the plots below overlay all five optimizers on the same axes at a fixed learning rate of 0.01.

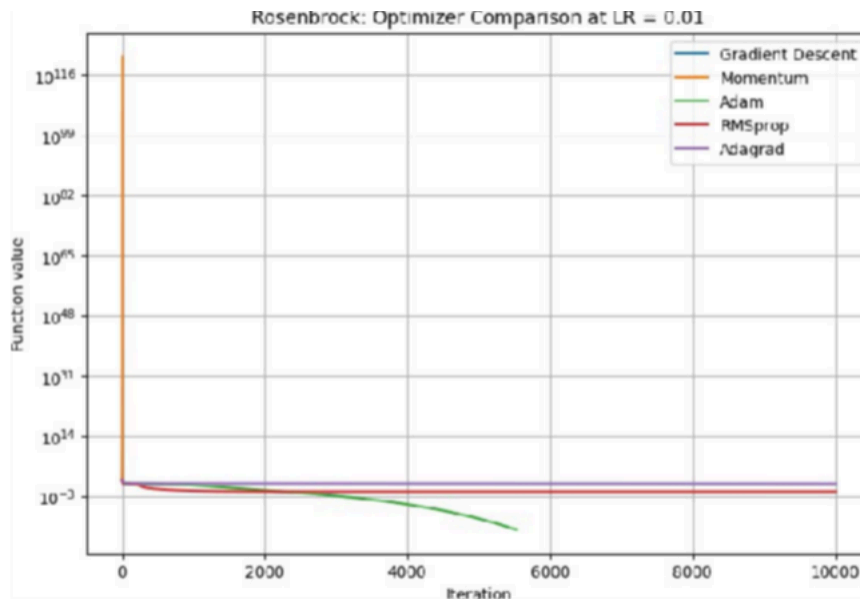


Figure A. Rosenbrock: all optimizers at LR = 0.01. Adam separates clearly from the rest, continuing to descend for thousands of iterations while Gradient Descent and Momentum diverge almost immediately.

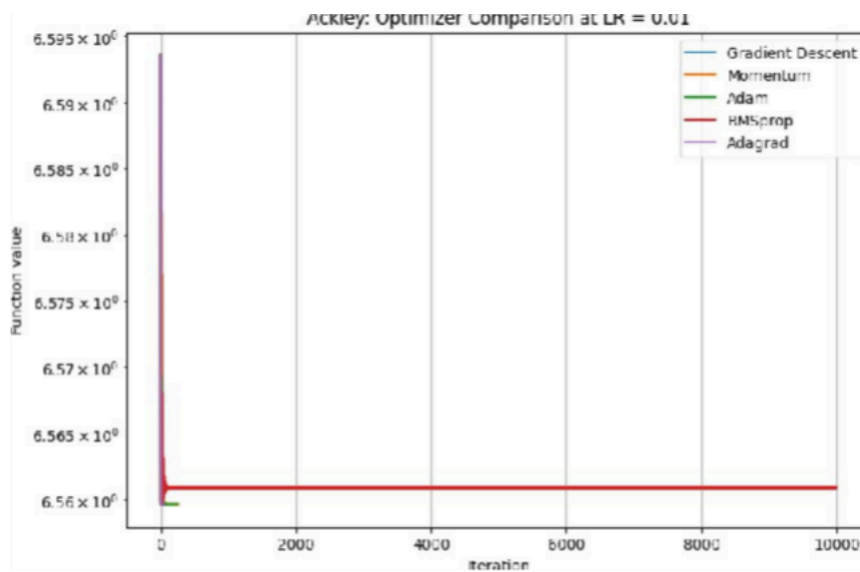


Figure B. Ackley: all optimizers at LR = 0.01. Most optimizers settle quickly into the same local minimum; differences show up mainly in how many iterations each needs to get there.

6. Best Results Summary

Ranking every finite result by final function value highlights the strongest optimizer / learning-rate combinations for each function.

6.1 Rosenbrock — Top 5 by Final $f(x)$

Optimizer	LR	Final $f(x)$	Solution	Status
Adam	0.1	7.70e-14	0.999999, 0.999999	converged
Adam	0.05	1.39e-13	0.999996, 0.999993	converged
Adam	0.01	3.10e-13	0.999994, 0.999989	converged
Adagrad	0.1	6.27e-04	0.974974, 0.950495	max_iter
RMSprop	0.01	2.22e-02	0.990144, 0.965534	max_iter

Table 3. Best Rosenbrock results, ranked by final function value (lower is better).

6.2 Ackley — Top 5 by Final $f(x)$

Optimizer	LR	Final $f(x)$	Solution	Status
Gradient Descent	0.05	3.2716	0.315579	max_iter
Gradient Descent	0.1	3.5955	0.947334	max_iter
Gradient Descent	0.01	6.5596	1.974452	converged
Adagrad	0.05	6.5596	1.974452	converged
Adagrad	0.1	6.5596	1.974452	converged

Table 4. Best Ackley results, ranked by final function value (lower is better). Note these are the best $f(x)$ values achieved, not global-optimum solutions.

7. Discussion and Conclusions

- Adam was the most robust optimizer overall: it converged reliably on both functions, avoided the numerical blow-up that affected plain Gradient Descent and Momentum on Rosenbrock, and reached the lowest function values on Rosenbrock across all learning rates tested.
- Fixed-step methods (Gradient Descent, Momentum) are highly sensitive to the curvature of the loss surface. On the steep, narrow Rosenbrock valley they diverged outright; on the flatter Ackley function they were stable and, at moderate-to-large learning rates, even outperformed the adaptive methods by escaping the nearest local minimum.
- Adaptive per-parameter methods (Adagrad, RMSprop) prevented divergence on Rosenbrock but converged slowly, and on Ackley their shrinking effective step size caused them to settle into the first local minimum reached rather than continuing to explore.
- Learning rate had opposite effects on the two functions: larger learning rates generally sped up convergence on Rosenbrock (for Adam) but pushed Momentum into instability on Ackley — reinforcing that a single learning rate is not universally "best" and should be validated per problem, as the experimental data — rather than assumption — confirms.
- A key takeaway from the Ackley results is that reaching "converged" status (small step size) is not the same as reaching the global optimum — several optimizers converged confidently to a local minimum at $x \approx 1.974$ rather than the true minimum at $x = 0$.

7.1 Limitations

- The Rosenbrock experiments for Gradient Descent and Momentum triggered numerical overflow warnings (RuntimeWarning: overflow encountered in scalar power); results are reported as "diverged" with infinite final function values.
- Results depend on the fixed starting points specified by the assignment ((-1.2, 1.0) for Rosenbrock, 2.0 for Ackley) and the fixed random seed (42); different initializations could change which local minimum is reached, particularly for the multimodal Ackley function.
- A maximum of 10,000 iterations and a step-size tolerance of $1e-8$ were used throughout; optimizers reported as "max_iter" may continue to improve slowly beyond this budget.

Assignment 2 - Task 2: Linear Regression via a Multi-Layer Neural Network Implemented from Scratch

1. Summary

This report documents and analyzes a Jupyter notebook (assignment2.ipynb) in which a feed-forward neural network is implemented entirely from first principles using NumPy — including manual forward propagation, manual backpropagation, and three hand-coded optimizers (vanilla gradient descent, momentum, and Adam). The network is trained to predict median house value in the California Housing dataset using two input features: median income (MedInc) and house age (HouseAge).

Two controlled experiments are run: (1) a learning-rate sweep under vanilla gradient descent, and (2) a comparison of the three optimizers at a fixed learning rate. The best-performing configuration (Adam, $\text{lr} = 0.01$, 1000 epochs) achieves a test MSE of 0.6652 (RMSE ≈ 0.816 , i.e. average prediction error of roughly \$81,600 given the dataset's \$100,000 target units). The report walks through the methodology, reproduces the key results in tabular form, reproduces the notebook's figures, and closes with a critical discussion of the model's limitations and concrete recommendations for improving it.

2. Objective and Scope

The assignment's implicit goal is to demonstrate a working understanding of the mechanics underlying neural network training — specifically:

- Implementing forward propagation through a multi-layer perceptron by hand.
- Implementing and comparing multiple optimization algorithms (SGD, Momentum, Adam) from their mathematical update rules.
- Evaluating the practical effect of the learning rate and the choice of optimizer on convergence behavior and generalization.

Everything in the notebook is built with NumPy and Matplotlib only — no PyTorch, TensorFlow, or scikit-learn estimator is used for the model itself; scikit-learn is used solely to fetch the dataset.

3. Dataset and Preprocessing

3.1 Data Source

The notebook loads the California Housing dataset via `sklearn.datasets.fetch_california_housing(as_frame=True)`. This is the standard 1990 California census dataset containing 20,640 rows, each representing a census block group. The full dataset offers eight predictive features (MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude), but this notebook narrows the input space to just two:

- MedInc — median income of the block group (in tens of thousands of USD)
- HouseAge — median age of houses in the block group

The regression target, MedHouseVal, is the median house value for the block group, expressed in units of \$100,000.

3.2 Train/Test Split

A manual 80/20 split is performed using a seeded random permutation of row indices (`np.random.seed(42)`), giving 16,512 training rows and 4,128 test rows. Using a fixed seed makes the split — and therefore every downstream comparison — reproducible, which is good experimental practice.

3.3 Feature Standardization

Both input features are standardized (zero mean, unit variance) using the mean and standard deviation computed only on the training set, and those same statistics are then applied to the test set. Standardization is particularly important here because MedInc and HouseAge live on very different numeric scales, which would otherwise distort gradient magnitudes across input dimensions.

The target variable MedHouseVal is not scaled. This is a valid choice — it keeps the loss directly interpretable in the target's native units — but it does mean the raw MSE values in this report are on the scale of $(100,000 \text{ USD})^2$ rather than a normalized, architecture-independent quantity.

4. Model Architecture

The network is a fully-connected feed-forward MLP with two hidden layers, implemented manually:

Layer	Shape	Activation
Input	2 features	—
Hidden 1	$2 \rightarrow 5$	ReLU
Hidden 2	$5 \rightarrow 3$	ReLU
Output	$3 \rightarrow 1$	Linear (identity)

Weights are initialized as small Gaussian random values (standard normal $\times 0.1$) and biases are initialized to zero, using a seeded NumPy generator (`np.random.default_rng(seed)`) so that every experiment starts from an identical set of initial weights — this is important, because it means the differences observed between runs are attributable to the learning rate or optimizer, not to a lucky/unlucky initialization.

The output layer has no activation function, which is the correct design choice for a regression task where the target (house value) is a continuous, unbounded-in-principle quantity.

5. Forward and Backward Pass

5.1 Forward Propagation

The `forward()` function computes, for a batch X : $Z1 = X \cdot W1 + b1$, $A1 = \text{ReLU}(Z1)$; $Z2 = A1 \cdot W2 + b2$, $A2 = \text{ReLU}(Z2)$; and the final prediction $Z3 = A2 \cdot W3 + b3$ (flattened to a 1-D vector). Intermediate activations are cached for reuse in backpropagation — a standard and efficient design that avoids redundant recomputation.

5.2 Loss Function

The loss is Mean Squared Error, $\text{MSE} = \text{mean}((y_{\text{true}} - y_{\text{pred}})^2)$, the natural default for continuous-valued regression targets.

5.3 Backpropagation

The `backward()` function derives gradients analytically via the chain rule: the output-layer error signal $dZ3 = (2/m) \cdot (y_{\text{pred}} - y_{\text{true}})$ follows directly from differentiating MSE with respect to the linear output. This is propagated backward through each layer, multiplying by the transposed weight matrix of the layer ahead and masking by the ReLU derivative (1 where the pre-activation was positive, 0 otherwise) at each hidden layer. Weight gradients are computed as the previous layer's activation transposed, matrix-multiplied with the downstream error signal; bias gradients are the column-sum of the error signal.

6. Optimizers Implemented

Three optimizers are implemented from their update equations, selectable via a single `update_params()` dispatch function:

Optimizer	Update Rule (per parameter)	Notes
Gradient Descent (gd)	$\theta \leftarrow \theta - lr \cdot \nabla \theta$	Plain full-batch steepest descent
Momentum	$v \leftarrow \beta_1 v + (1 - \beta_1) \nabla \theta$; $\theta \leftarrow \theta - lr \cdot v$	$\beta_1 = 0.9$; exponential moving average of gradients
Adam	bias-corrected 1st & 2nd moment estimates; $\theta \leftarrow \theta - lr \cdot \hat{v} / (\sqrt{\hat{s}} + \epsilon)$	$\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$, with bias correction via time step t

The Adam implementation correctly includes bias correction (dividing the raw moment estimates by $1 - \beta^t$), which matters most in the first few iterations when the moving averages are still close to their zero initialization. All optimizer state (velocity v , squared-gradient accumulators, and time step t) is initialized per-run inside `init_optimizer_state()`, so each experiment starts cleanly.

Training itself (the `train()` function) uses full-batch gradient descent — every epoch computes the gradient over all 16,512 training examples at once, rather than mini-batches. This is a reasonable simplification for an educational assignment and keeps the optimizer comparison clean (no mini-batch sampling noise to confound the results), though it does not reflect how these optimizers are typically used in practice on larger datasets (see Section 8).

7. Experimental Results

7.1 Experiment 1 — Effect of Learning Rate

With the optimizer fixed to plain gradient descent and training run for 1000 epochs, two learning rates were compared:

Learning Rate	Final Train MSE	Test MSE	Test RMSE ($\approx$ \$)
0.01	0.6377	0.6655	0.816 ($\approx$ \$81,600)
0.001	1.3981	1.4180	1.191 ($\approx$ \$119,100)

Assignment 2 — Neural Network Analysis Report

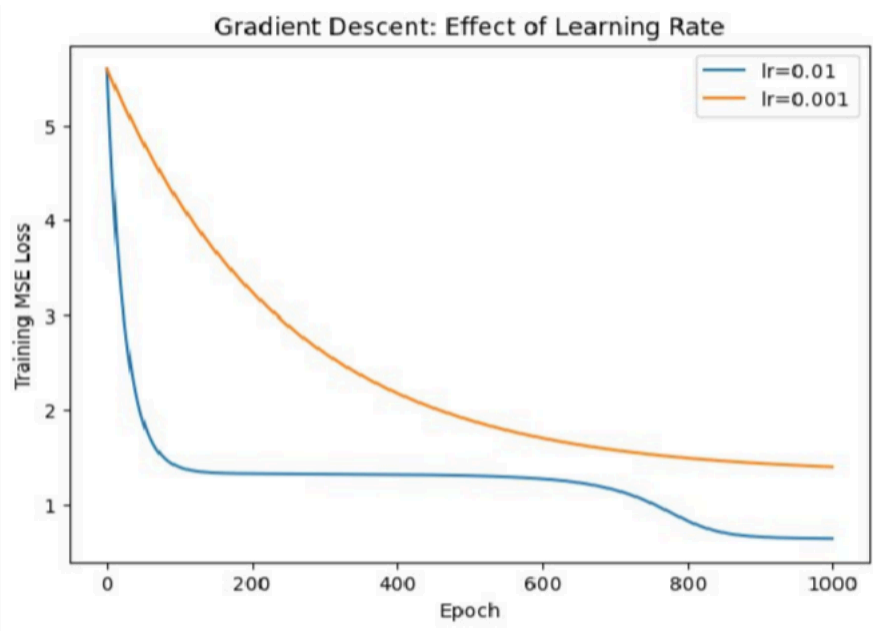


Figure 1. Training MSE vs. epoch for $lr = 0.01$ and $lr = 0.001$ (vanilla gradient descent, 1000 epochs).

The higher learning rate (0.01) converges substantially faster and reaches a lower loss plateau (~ 0.64) within roughly 400–500 epochs. The lower learning rate (0.001) is still descending steadily at epoch 1000 and has not converged — it reaches only 1.40, more than double the final loss of the $lr = 0.01$ run. This is the expected, textbook relationship between step size and convergence speed for a smooth, well-conditioned loss surface: 10× smaller steps roughly correspond to needing a similar order-of-magnitude more iterations to reach the same point, and 1000 epochs simply isn't enough at $lr = 0.001$. Neither curve shows divergence or oscillation, indicating both learning rates are within the stable range for this architecture.

7.2 Experiment 2 — Optimizer Comparison

With the learning rate fixed at 0.01 and training run for 1000 epochs, all three optimizers were compared:

Optimizer	Final Train MSE	Test MSE	Test RMSE ($\approx$ \$)
Gradient Descent	0.6377	0.6655	0.816 ($\approx$ \$81,600)
Momentum	0.7003	0.7215	0.849 ($\approx$ \$84,900)
Adam	0.6396	0.6652	0.816 ($\approx$ \$81,600)

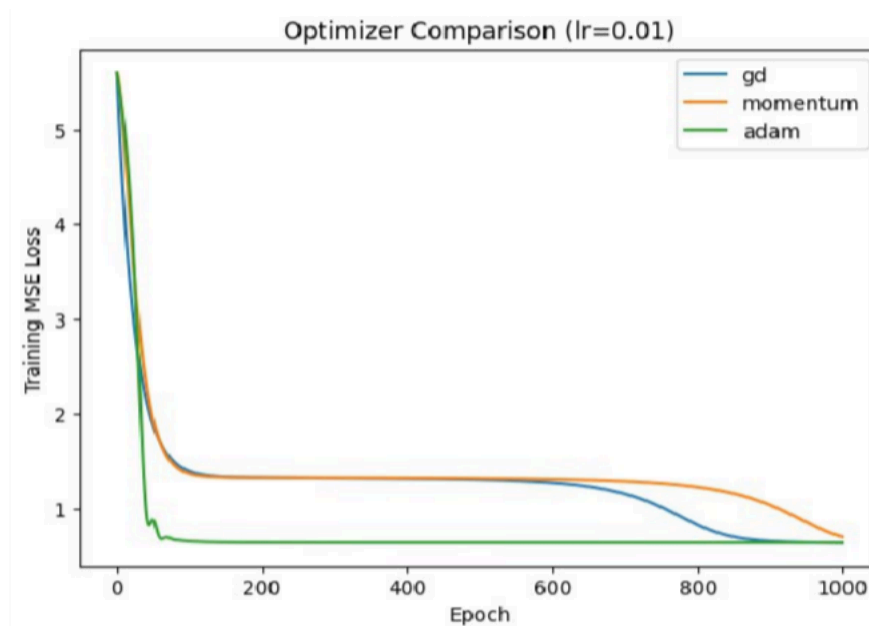


Figure 2. Training MSE vs. epoch for GD, Momentum, and Adam, all at $lr = 0.01$.

Adam converges dramatically faster than the other two methods — it reaches its final loss plateau within roughly 100 epochs, an order of magnitude faster than plain GD. This matches Adam's well-known behavior: its per-parameter adaptive step sizes let it take large effective steps early in training even when raw gradients are small, and the bias-corrected second-moment term keeps updates well-scaled from the very first iterations.

Momentum is the notable underperformer here, finishing with both the highest train loss (0.7003) and the highest test loss (0.7215) of the three methods — worse than even plain gradient descent at the same nominal learning rate. with $\beta_1 = 0.9$, momentum's effective step size is roughly $1/(1-\beta_1) = 10\times$ the raw learning rate once the velocity term has built up. Because this is deterministic full-batch training (no mini-batch gradient noise for momentum to smooth over, which is the setting where momentum typically helps most), that amplified, inertial step size makes momentum prone to overshooting and oscillating around the minimum rather than settling into it — consistent with the visibly bumpier curve in Figure 2. In other words, at $lr = 0.01$, momentum is effectively operating with a much larger implicit learning rate than the plain-GD and Adam runs it's being compared against.

Plain GD and Adam ultimately reach nearly identical final losses (0.6377 vs. 0.6396 train; 0.6655 vs. 0.6652 test) — Adam just gets there far faster. This is a useful and often-overlooked insight: for a small, well-conditioned problem like this one, adaptive optimizers mainly buy convergence speed, not a better final solution.

7.3 Final Model Evaluation

The Adam-trained model (chosen as the best performer) was evaluated on the held-out test set, yielding Test MSE = 0.6652 (RMSE $\approx$ 0.816, roughly \$81,600 average absolute-scale error against a target measured in \$100,000 units).

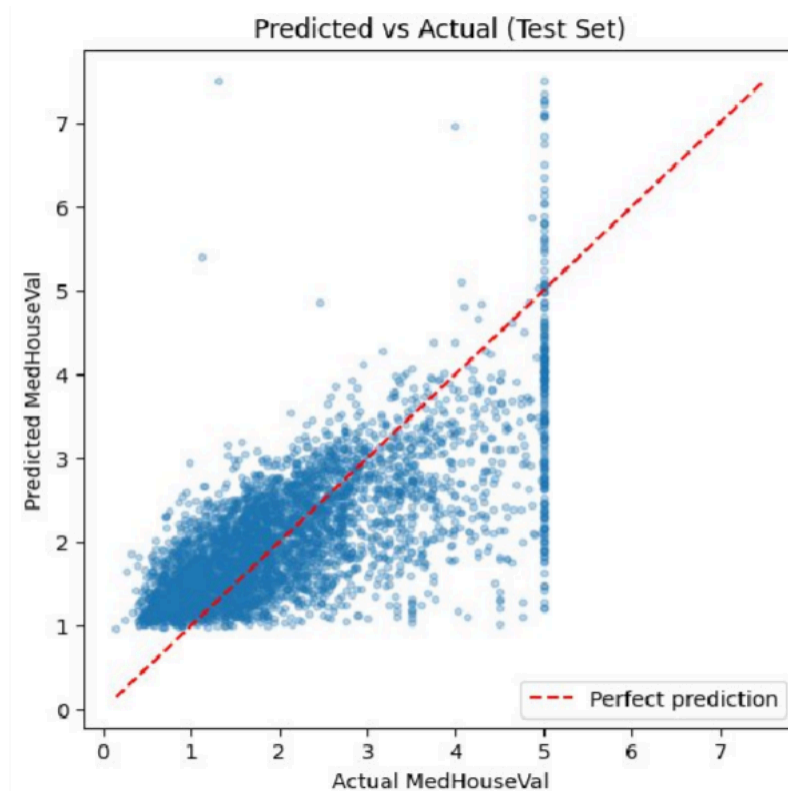


Figure 3. Predicted vs. actual median house value on the test set, with the $y = x$ reference line indicating perfect prediction.

The scatter plot shows a positive, roughly linear relationship between predicted and actual values — the model has clearly learned a meaningful signal from income and house age. Two patterns are worth calling out specifically:

- Systematic spread around the diagonal: even at any fixed actual value, predictions vary considerably, which is expected given that only 2 of the dataset's 8 features are used — a large share of the variance in house price (driven by location, room count, occupancy, etc.) is simply not visible to this model.
- A vertical band of points at actual = 5.0: this is not a modeling artifact but a known property of the California Housing dataset itself — MedHouseVal is top-coded (censored) at \$500,000 in the original census data, so all block groups above that threshold are recorded as exactly 5.0. The model, having no way to represent this artificial ceiling, predicts a range of values for these capped block groups (some far above 5.0, some below), which is the correct behavior for a regression model but does mean test error is partly inflated by an unlearnable data artifact rather than a modeling deficiency.

8. Discussion, Limitations

8.1 Strengths of the Implementation

- Backpropagation is derived and implemented correctly and matches the analytical gradients for this architecture.
- The experimental design is controlled well: identical seeded initialization across runs isolates the effect of the variable being studied (learning rate, then optimizer).
- Preprocessing correctly fits standardization statistics on the training set only, avoiding test-set leakage.
- The Adam implementation includes bias correction, which is often omitted in simplified student implementations.

8.2 Limitations

- Feature scope: using only MedInc and HouseAge discards 6 of 8 available predictive features (notably AveRooms, Latitude/Longitude, and AveOccup, which are typically strong predictors of California housing prices). This is very likely the dominant factor capping test performance around $MSE \approx 0.64\text{--}0.67$, not the choice of optimizer.
- Full-batch training only: no mini-batching is used, so the momentum and Adam comparisons don't reflect the stochastic-gradient regime in which these optimizers are most commonly deployed and where their noise-averaging properties matter most.
- No validation set for hyperparameter tuning: the learning rate and optimizer choices are compared directly on the test set, which risks a mild optimistic bias if these results were used to "pick a winner." A held-out validation split would be more rigorous.
- Single train/test split: results rest on one 80/20 split with a fixed seed; a k-fold cross-validation would give a more robust estimate of generalization error and its variance.
- No regularization: with only 2 inputs and a small ($2 \rightarrow 5 \rightarrow 3 \rightarrow 1$) network, overfitting is unlikely to be severe here, but no weight decay, dropout, or early stopping is implemented, so this hasn't been explicitly verified.
- Weight initialization: small-Gaussian ($\times 0.1$) initialization works adequately here given the network's small size, but is not the same as principled ReLU-aware initialization (e.g., He initialization), which becomes more important for deeper networks.
- Target left unscaled: MedHouseVal is not standardized, which is a reasonable interpretability choice but means the reported MSE values are unit-dependent and not directly comparable to MSE from a model trained on standardized targets.

9. Conclusion

This notebook demonstrates a correct, from-scratch implementation of a small feed-forward neural network, manual backpropagation, and three optimization algorithms (GD, Momentum, Adam), evaluated through two well-controlled experiments. The results are internally consistent with optimization theory: higher learning rates converge faster (until instability would set in), and Adam reaches a good solution far faster than plain GD or Momentum, though — in this deterministic, full-batch, low-dimensional setting — it does not reach a meaningfully better final solution than plain GD. The dominant limitation is not the optimization machinery, which is implemented soundly, but the restricted two-feature input space, which caps achievable predictive accuracy well below what the full California Housing feature set would allow.

Run	Optimizer	Learning Rate	Epochs	Final Train MSE	Test MSE	Test RMSE
1	GD	0.01	1000	0.6377	0.6655	0.816
2	GD	0.001	1000	1.3981	1.4180	1.191
3	GD	0.01	1000	0.6377	0.6655	0.816
4	Momentum	0.01	1000	0.7003	0.7215	0.849
5	Adam	0.01	1000	0.6396	0.6652	0.816